Build Steps

Part 1 of 5 · Set up your agent in the new builder

1 Create your agent

Open Copilot Studio (copilotstudio.microsoft.com), select **Create**, then **New agent**. In the redesigned builder everything lives under the **Build** tab, with a settings panel down the right. Set:

Agent name Sales Pipeline Copilot
Description Sellers and managers get pipeline insight, deal-health flags, and in-line CRM actions without leaving Teams.
Solution / Environment Sales Solution · Default environment
Icon Use a recognisable colour and emoji, or upload a brand icon

2 Choose your model

Open the **Model** selector at the top of the right-hand panel. Copilot Studio now lets you pick the model your agent reasons with. This build uses **Claude Sonnet 4.6**; you can switch later without rebuilding.

Reasoning is built in

- The agent reasons over your instructions, knowledge, skills and tools, and decides what to do each turn
- Handles paraphrases, multi-step asks and follow-ups
- Blends a knowledge lookup with a tool call in one reply
- Calls the right skill when intent matches

Choosing a model

- Claude Sonnet 4.6: strong reasoning, a good default here
- Lighter models: lower latency and cost for simple, high-volume flows
- Switch the model and re-run the same evaluation set to compare
- Check data residency and model availability for your tenant

3 Write your instructions

The **Instructions** box is the heart of the Build tab. Define the agent's role, audiences and rules here. The agent reasons over these together with everything in the right-hand panel. Paste:

```
You are the Sales Pipeline Copilot. You help two audiences:
1) Account executives reviewing their own deals.
2) Sales managers reviewing their team or region.

Rules:
- Use the uploaded opportunity data as your single source of truth.
- For analytical questions, calculate from the rows and explain method.
- For deal updates (stage, close date, next step), call the CRM tool.
- Flag stalled deals (no activity over 21 days) and slipping close dates.
- Never quote pipeline numbers across regions a user does not own.
```

Build Steps

Part 2 of 5 · Ground it in your data

4 Add Knowledge

Open **Knowledge** in the panel, select **Add knowledge**, then **File**, and upload **Sales_Pipeline_Data.xlsx**. Knowledge gives the agent trusted context to guide its answers. This file contains:

- Account details (account name, industry, region)
- Deal details (stage, amount, probability, product line, lead source)
- Ownership (account executive)
- Time data (expected close date, last activity date, days in stage)

Layer more sources later: SharePoint sites, Dataverse, Microsoft Graph connectors, or public web.

5 Bring in Microsoft IQ

Open **Microsoft IQ** in the panel. Microsoft IQ brings in work context, business data and app signals from across your tenant, so the agent grounds answers in more than the file you uploaded.

For this agent, Microsoft IQ brings in account activity from Outlook and Teams alongside the linked Dataverse account, so deal-health reflects real engagement, not just the CRM stage. Microsoft IQ honours each user's existing permissions, so the agent only surfaces what the signed-in user is already allowed to see.

Build Steps

Part 3 of 5 · Give it skills and tools

6 Add a Skill

Open **Skills** in the panel, then **Add a skill**. A skill defines a reusable behaviour through structured instructions. It is the new home for the conversational flows you used to build as topics: the agent calls the skill when a user's intent matches.

Skill name: Update Deal Stage
Description: Lets a seller move an opportunity to a new stage and capture the next step in one conversational turn.
When to use it (intent hints the agent reasons over):

- Move Cobalt Insurance to Negotiate
- Update the stage on deal OPP3015
- Set next step on the Northwind deal
- Progress my deal to Propose

Structured steps:

- Identify the opportunity by name or ID, and confirm if ambiguous
- Ask for the new stage and the next step
- Look up the current stage from knowledge and show the change
- Call the 'Update Opportunity' tool with the new values
- Confirm the update succeeded and offer to draft a follow-up email

7 Add a Tool

Open **Tools**, then **Add a tool**. **Skills decide what to do; tools take the action.** A tool is a typed, callable action with named inputs and outputs that the model fills from the conversation. Choose one of: a **prebuilt connector** (Outlook, Teams, SharePoint, Dataverse, ServiceNow and more), a **Power Automate cloud flow**, a **custom HTTP request**, or a **Model Context Protocol (MCP) server**. This build uses a Power Automate flow over the Dataverse opportunity table, so one call can update the record, stamp the next step, and return the new timestamp.

Tool name: Update Opportunity
Type: Power Automate flow that updates a row on the Dataverse 'opportunity' table and logs the change
Description (the model reads this to decide when to call): Updates stage, close date, and next step on a CRM opportunity record. Use for any in-line deal update.
Inputs the agent collects or infers:

- opportunityId — text, the CRM record key, e.g. OPP3015
- newStage — text, one of Qualify / Discover / Propose / Negotiate / Closed Won / Closed Lost
- expectedCloseDate — date, the revised close date
- nextStep — text, the agreed next action in the seller's words

Outputs returned to the agent:

- updateStatus — text, e.g. Success
- lastModified — text, e.g. 2026-06-08T14:21:05Z
- recordUrl — text, deep link to the opportunity in CRM

7.1 · Build the cloud flow that does the work

From inside Copilot Studio choose **Tools > Add a tool > New > Power Automate**. This opens the flow designer pre-wired with the Copilot Studio trigger and, crucially, keeps the flow in the **same environment** as your agent (Sales Solution · Default). An environment mismatch is the most common reason a finished tool never appears in the picker, so confirm the environment switcher (top-right) before you build.

Trigger — *When an agent calls the flow*. Add four input parameters, named exactly: `opportunityId` (text), `newStage` (text), `expectedCloseDate` (date), `nextStep` (text).

– **Action 1 – update the record.** Dataverse *Update a row* on the **opportunity** table. Row ID = opportunityId; map `stepname` = newStage, `estimatedclosedate` = expectedCloseDate, and a `nextstep` custom column = nextStep.
– **Action 2 – log the change.** Dataverse *Add a new row* to a **Deal Activity** table capturing the opportunityId, the new stage, and `utcNow()`, so managers get an audit trail of in-line edits.
– **Action 3 – build the deep link.** *Compose* `recordUrl` = `concat('https://org.crm.dynamics.com/main.aspx?etn=opportunity&id=', opportunityId)` so the agent can hand the seller a click-through.
– **Final action – return to the agent.** Add *Respond to the agent* (Return value(s) to Copilot Studio) with outputs updateStatus = 'Success', lastModified = `body('Update_a_row')?['modifiedon']`, recordUrl = the composed link.
– **Save** the flow as `Sales – Update Opportunity`.

7.2 · Register and map the tool in your agent

– Back in Copilot Studio the flow is attached automatically; if you built it separately, **Tools > Add a tool > Flows** and select it.
– Set the display name to **Update Opportunity** and write the **Description** for the model — it is what the agent reads to decide when to call the tool.
– For each input set **Fill using**: opportunityId and nextStep → **Fill with AI** from the conversation; newStage → **Fill with AI** but add an input description listing the six valid stages so the model never invents one; expectedCloseDate → **Fill with AI**, noting it should resolve relative dates like 'next Friday'.
– Confirm `updateStatus` and `recordUrl` are returned to the agent so the skill can confirm the change and offer the deep link, rather than just claiming success.
– **Connection & auth:** set the Dataverse connection the flow uses to run as the signed-in user. The connector chosen here is exactly what your DLP policy must permit (see Step 10).

7.3 · Worked example — what flows in and out

When a seller says *"move the AlpenTech deal to Propose, close date next Friday"*, the model fills the inputs and calls the tool with this payload:

```
{
  "opportunityId":     "OPP3042",
  "newStage":          "Propose",
  "expectedCloseDate": "2026-06-12",
  "nextStep":          "Send pricing proposal for review"
}
```

The flow updates the row, logs the activity, and returns:

```
{
  "updateStatus": "Success",
  "lastModified": "2026-06-08T14:21:05Z",
  "recordUrl":    "https://org.crm.dynamics.com/main.aspx?etn=opportunity&id=OPP3042"
}
```

The agent then replies in natural language, quoting the real result: *"Done — AlpenTech is now at Propose with a close date of 12 Jun and next step 'Send pricing proposal for review'. Open the record if you want to review it."*

7.4 · Test the tool before you trust it

– **In isolation:** open the flow in Power Automate, choose *Test* > *Manually*, supply the four inputs, run it, and confirm the opportunity row changes stage, a Deal Activity row is logged, and the run returns all three outputs.
– **End to end:** in **Preview** (Step 11) type "move Cobalt Insurance to Negotiate" and check the activity view shows Update Opportunity firing with the inputs the model inferred and the timestamp it read back.

Build Steps

Part 4 of 5 · Extend, remember and govern

8 Add Connected agents

Open **Connected agents**. A connected agent lets this agent collaborate with others, delegating specialist work and sharing context across them (agent-to-agent). Keep each agent focused on what it does best.

Connect a **Proposal Drafting Agent** that turns an updated opportunity into a first-draft quote or statement of work, so a stage change can flow straight into the next deliverable.

9 Turn on Memory

Toggle **Memory** on. Memory lets the agent remember interactions, workflows and context across sessions, so returning users do not have to repeat themselves. For this agent it remembers:

- A seller's book and preferred pipeline view, so 'show my pipeline' opens correctly
- Deals the user updated recently, so follow-ups need no re-stating

Memory is scoped to each user and follows your retention and DLP policy. Review what the agent retains before you publish.

10 Govern your agent

Open the agent settings and lock these down before real users arrive:

Authentication: use *Authenticate with Microsoft (Entra)* so the agent runs as the signed-in user and inherits their permissions.
Content moderation: set the moderation level (start at Medium) and review it for your audience.
Data loss prevention: confirm your DLP policy covers every connector your tools use — for Update Opportunity that means the Dataverse connector must sit in the right DLP group, or the tool is blocked at runtime.
Channels and access: review which channels are enabled and who can use the agent.

Build Steps

Part 5 of 5 · Preview, evaluate and publish

11 Preview your agent

Switch to the **Preview** tab to chat with your agent as you build. Open the **activity view** on any response to inspect:

- Which skill, tool or knowledge source fired, and in what order
- The inputs the agent collected and the exact payload each tool returned
- Citations linking each answer back to a row of your data
- Latency per step, useful when tuning multi-tool flows

12 Evaluate

Open the **Evaluate** tab, then **New evaluation**. Build a test set so you can measure quality before and after every change. Score each response on **groundedness** (did it stay inside the knowledge), **relevance** (did it address the question), and **accuracy** (did it match the expected outcome).

Test set: Sales Pipeline Copilot · Baseline v1
Input: Show me my pipeline by stage with weighted value
Expect: Calculates weighted value as amount times probability per row, sums by stage.
Input: Which deals haven't had activity in 21 days?
Expect: Returns rows where Last Activity Date is more than 21 days before today.
Input: Move Cobalt Insurance to Negotiate and set next step to 'Procurement review'
Expect: Triggers the Update Deal Stage skill and calls the Update Opportunity tool.

13 Publish and Monitor

Click **Publish**, then enable the channels your users live in: Microsoft Teams, Microsoft 365 Copilot, a custom website embed, or other channels. Teams and M365 Copilot need tenant admin approval, so plan for that lead time. After publishing, open the **Monitor** tab to watch sessions, resolution and escalation outcomes, and quality trends over time, and to see where users drop off.

Try These Live

Sample queries to test your agent during the build-along.

Pipeline Health
Show pipeline by stage with total and weighted value. · What is the average days-in-stage by stage?

Risk & Slippage
List opportunities where Last Activity Date is more than 21 days ago. · Show deals with close date in the past that are still open.

In-line CRM Actions
Move the AlpenTech deal to Propose, close date next Friday.
Set next step on the Northwind Bank deal to 'Legal redline this week'.

How to Think About This Agent

This agent replaces	With
• Hunting through CRM dashboards • Switching to CRM just to update a stage • Manual weighted forecast roll-ups	✓ Pipeline insight where sellers already work ✓ CRM updates in a single conversational turn ✓ Sharper forecast conversations with managers

Trusted, governed, enterprise AI.
Built once, used everywhere.

Tip for real-world use
Use Dataverse-native security roles so sellers see only their own book, managers see their team, and execs see the region. The agent inherits those permissions automatically, with no extra access logic to maintain.

You've built a Copilot Studio agent ✓
You now have an AI agent that can:

- Answer pipeline questions
- Update CRM in conversation
- Surface stalled and slipping deals
- Lift sellers' time-on-selling